

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

A broad wave of recent progress in deep learning optimization has produced a family of algorithms—including Schedule-Free optimizers, AdEMAMix, MARS, and Lion—that each rethink the classical momentum mechanism at the heart of modern optimizers. In a parallel line of theoretical work, acceleration of stochastic gradient descent (SGD) in noise-dominated regimes has been obtained by decoupling the momentum coefficient from the weight assigned to the current gradient. In this paper, we establish explicit and general connections between these two lines of work, showing that a wide range of contemporary optimizers can be understood through the unifying lens of accelerated SGD. We substantiate these theoretical findings with representative experiments on a language modeling task. Across these settings we find that AdEMAMix, the method that most closely mirrors accelerated versions of stochastic gradient descent, consistently exhibits the strongest performance. Building on these insights, we introduce a modification to AdEMAMix, termed Simplified-AdEMAMix, which preserves the performance of AdEMAMix across the full range of batch-size settings we study—from small to large—while eliminating the need for two separate momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

A steady stream of optimization algorithms has recently been introduced for deep learning, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these optimizers were proposed from very different starting points and for a range of practical motivations, they share a common structural feature that cuts across their individual designs: each reshapes the momentum scheme used in optimization. This shared characteristic suggests that they may be better understood not as isolated methods but as members of a broader design space.

A separate and largely independent body of theoretical research has focused on accelerating gradient descent in noisy environments. While classical momentum methods, such as heavy-ball or Nesterov momentum, suffice to accelerate deterministic gradient descent (particularly for quadratic objectives), they do not accelerate SGD in general (Jain et al., 2018; Liu & Belkin, 2020). This limitation has motivated a family of alternative momentum schemes designed to recover acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). A unifying observation is that, despite their differing derivations, all of these accelerated SGD methods can be interpreted through a single principle: decoupling the momentum coefficient from the weight assigned to the current gradient in the optimizer update.

Our primary contribution is to bridge these two research directions with a general and explicit correspondence. Specifically, we show that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging. More broadly, we argue that a whole class of contemporary optimizers—including Lion, Schedule-Free AdamW, and AdEMAMix—can be understood as combining preconditioning with accelerated SGD. While isolated aspects of these connections have surfaced in prior literature (Defazio, 2021), to the best of our knowledge the general relationship between this emerging class of optimizers and accelerated SGD has not previously been made explicit.

To validate these theoretical insights, we conduct experiments on a decoder-only transformer trained for language modeling in a small-batch, noise-dominated regime that is representative of the setting in which acceleration is expected to matter. Consistent with our analysis, the behavior of Schedule-Free AdamW closely tracks that of accelerated SGD-based AdamW (Algorithm 3), and accelerated methods offer a modest but consistent advantage at small batch sizes. Just as importantly, we show that these benefits taper off as the batch size grows large, exactly as the connection

to accelerated SGD predicts—evidence that the framework accounts not only for where these methods help but also for where they should not.

Our main contributions are stated below:

1. We establish precise and broadly applicable theoretical connections between accelerated SGD and a family of recently proposed optimizers, including Schedule-Free SGD and AdEMAMix, placing methods with disparate motivations within a single framework.
2. We provide empirical validation on a decoder-only transformer, comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS. Across these methods, AdEMAMix—the optimizer that aligns most closely with accelerated SGD variants—consistently emerges as the strongest performer.
3. As anticipated from the equivalence to accelerated SGD, the advantages of these methods fade at large batch sizes relative to Adam. In particular, we show that Adam with momentum scheduling alone can match the performance of AdEMAMix, illustrating the explanatory reach of the framework across batch-size regimes.
4. At high batch sizes, Schedule-Free AdamW performs markedly worse than AdamW with cosine decay, a gap we trace to the intrinsic coupling of momentum and weight averaging coefficients that is characteristic of Schedule-Free optimizers more generally.
5. We introduce a modification to AdEMAMix, termed Simplified-AdEMAMix, which preserves the performance of AdEMAMix across the full span of batch-size regimes we consider, from small to large, while eliminating the need for two distinct momentum terms.

2 RELATED WORK

We situate our work within two intertwined strands of the literature: the theory of accelerated SGD variants and the broader family of optimization algorithms that, whether by design or by discovery, share their structure.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares problems. Kidambi et al. (2018) subsequently simplified the update rule and formally established that plain momentum does not provide acceleration in this case. A line of follow-up work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) progressively broadened these results to general convex and strongly convex objectives under a variety of theoretical assumptions, steadily widening the class of problems for which acceleration under noise is understood.

Over the years, a diverse set of practical optimizers has emerged that, on closer inspection, echoes the accelerated SGD variants above. Lucas et al. (2019) proposed aggregating a weighted sum of several momentum terms, each with its own coefficient, to form the update. Ma & Yarats (2019) developed an optimizer explicitly inspired by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) arrived—via a genetic search over the space of update rules—at an optimizer that, like earlier accelerated SGD variants, assigns different weights to the gradient and the momentum term. In a related vein, Pagliardini et al. (2024) introduced a method that blends two distinct momentum timescales in the final update. Viewed together, these methods point to a recurring design motif that our framework aims to make explicit.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a well-established and widely used technique for accelerating the convergence of gradient descent across deterministic optimization settings. For weights w_t and a momentum coefficient β , the momentum update takes the general form:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a broadly applicable technique in stochastic optimization for reducing the noise carried by the iterates. Rather than returning the final iterate w_T , one computes a weighted average $\bar{w}_T$ of the iterates, with weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

All instances of weight averaging in this paper utilize coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

In this section, we lay out a single generalized framework that encompasses a wide range of accelerated SGD methods proposed across the literature:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are (possibly time-dependent) scalar coefficients, and g_t represents the stochastic gradient evaluated at w_t . We use the subscript ‘a’ to indicate coefficients that adhere to this specific accelerated SGD formulation.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum as a special case. More importantly, and as observed in prior work, the accelerated SGD algorithms proposed across the literature—including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—all fall directly within this single framework, which suggests it captures the essential structure shared by the family rather than any one method. A precise demonstration of this equivalence is provided in Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section, we establish precise theoretical connections between accelerated SGD and a broad set of existing optimizers, using Schedule-Free optimizers, Lion, MARS, and AdEMAMix as representative cases that span the design space. Building on these insights, we propose a simplified variant of AdEMAMix that relies on a single momentum term while retaining performance comparable to AdEMAMix across the full range of batch-size regimes we consider, from small to large.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant-learning-rate optimizer designed to remove the need for learning-rate scheduling altogether. Following the notation of Defazio et al. (2024), its update equations are:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t represents the current model weights (where the gradient is evaluated), while x_t denotes the weights used for evaluation.

We first express the update in terms of y_t and m_t , where we define:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Further simplifying m_{t+1} , we obtain:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. Given m_t , we now examine the update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. Consequently, y_t in Schedule-Free SGD precisely follows the accelerated SGD framework. However, x_t is used for evaluation in Schedule-Free SGD. We now analyze the dynamics of x_t :

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus, x_t is a weighted average of y_t , and recursively expanding x_t confirms that it is an exponential average of y_t whenever c_t is held constant. This establishes the general statement that Schedule-Free SGD can be understood as accelerated SGD followed by weight averaging—a decomposition that, as we discuss below, extends naturally to the adaptive (Adam-based) variants used in practice.

The benefits of Schedule-Free SGD can be attributed to two complementary components:

1. Improved performance compared to standard SGD with momentum, due to its equivalence to accelerated SGD.
2. The ability to use a constant learning rate without scheduling, enabled by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

We note two practical advantages that distinguish Schedule-Free SGD/Adam:

- It requires no additional memory for weight averaging.

- It removes the need to expose an explicit weight-averaging coefficient as a separate hyperparameter.

These advantages, however, come from tying momentum and weight averaging together, and in Section 5.1 we show that this coupling does not carry over gracefully to the large-batch regime.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), when $\beta = 0$ the general update collapses to standard SGD with weight averaging. Because $c_t = 1/t$ in this case, averaging is applied uniformly from the very first iterate.

4.1.2 CASE: $\beta = 1.0$

Likewise, as noted in (Defazio et al., 2024), when $\beta = 1$ the update reduces to standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$. These two limiting cases bracket the behavior of the method across its full range of β .

4.1.3 CASE: $\beta = 0.9$

For the intermediate and most commonly used setting $\beta = 0.9$, the default in Schedule-Free SGD, the general picture specializes as follows:

- As c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio of the weight assigned to the current gradient versus momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is applied approximately over the most recent 10% of the iterates.

4.2 LION

The update rule for Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) thus fits directly into our framework: it can be interpreted as an accelerated SGD method followed by a coordinate-wise sign operation, providing another instance of the same underlying structure.

4.3 MARS

We now show that the practical version of the recently proposed optimizer MARS (Yuan et al., 2024), referred to as MARS-Approx, likewise fits the accelerated SGD framework once a preconditioning step is included—further evidence that the framework reaches beyond any single optimizer. The update equations (ignoring bias correction and clipping) are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t represent the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Rewriting the update using $\hat{m}_t = m_t - \gamma g_t$, we obtain:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This rewriting makes clear that the momentum update again follows the general accelerated SGD framework, with the only substantive departure being that MARS adopts a preconditioning scheme distinct from AdamW’s. We examine the empirical consequences of this design choice in Section 5.

4.4 ADEMAMix

The recently proposed optimizer AdEMAMix (Pagliardini et al., 2024) is, among the methods we consider, the one that most closely mirrors accelerated SGD-based AdamW. Where accelerated SGD forms a linear combination of the current gradient and a single momentum term, AdEMAMix instead maintains two distinct momentum terms with different coefficients and combines them linearly. The algorithm is stated in full in Algorithm 1.

To make the correspondence transparent, we analyze the variant of AdEMAMix with $\beta_1 = 0$. As shown in Pagliardini et al. (2024), this variant performs nearly identically to the full version at small batch sizes, a finding our experiments in Section 5 independently reproduce. With $\beta_1 = 0$, AdEMAMix falls squarely within the general accelerated SGD framework (Equation (1)). Beyond this structural match, we further show that AdEMAMix’s prescribed schedules for the momentum coefficient β_3 and the relative-weight parameter α closely track the theoretical schedules derived for accelerated SGD (Gupta et al., 2023), so that the alignment holds not only in form but in the recommended hyperparameter dynamics.

In smooth convex optimization, achieving acceleration in stochastic settings requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately realizes this same scheme by scaling up β_3 accordingly, reinforcing that its design coincides with the theory rather than merely resembling it superficially.

Additionally, note that in accelerated SGD schemes, momentum is maintained in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas in Algorithm 1, the momentum update follows:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in m_t under accelerated SGD grows proportionally to t ; the coefficient α in AdEMAMix scales in exactly the same way. This shared growth behavior offers a principled explanation for why AdEMAMix tends to outperform the other optimizers we study, as observed in Figure 1.

The same framework also anticipates where AdEMAMix should struggle. At large batch sizes, AdEMAMix with $\beta_1 = 0.0$ exhibits a performance decline, as reported in Pagliardini et al. (2024). The theory of Gupta et al. (2023) indicates that, as the batch size grows, the weight placed on the current gradient must be reduced. AdEMAMix instead pins this weight at a fixed value of 1, and this mismatch with the theoretically prescribed behavior is a natural candidate

explanation for its degraded large-batch performance—illustrating that the connection to accelerated SGD is predictive across regimes, not only in the favorable one.

In the following section, we turn this diagnosis into a design: we introduce a simplified variant of AdEMAMix that places an explicit weight on the current gradient, removes the need to schedule α , and retains only a single momentum term. We then empirically confirm that this simplified version matches AdEMAMix across both small- and large-batch setups.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building on the insights above, we propose a simplified optimizer that dispenses with two separate momentum terms and with any need to schedule α . The method is presented in Algorithm 2, where we adopt theory-style momentum in place of the exponential-moving-average (EMA) style, and assign a fixed weight α to the gradient in the final update. The construction gracefully includes standard Adam as a special case: setting $\alpha = 0$ recovers Adam (up to appropriate transformations of η and β_1). In Section 5, we show that this single-momentum variant matches AdEMAMix across the range of batch sizes we study, from small to large.

5 EXPERIMENTS

To test our theoretical predictions in a realistic large-scale training setting, we run experiments on a 150-million-parameter decoder-only transformer trained for language modeling on the C4 dataset—a standard and widely used benchmark for this class of models. The model is trained with a sequence length of 1024 and a batch size of 32, over 15 billion tokens ($\approx 5 \times$ Chinchilla), which places training firmly in the noise-dominated regime where accelerated methods are expected to matter.

We compare the following optimization algorithms:

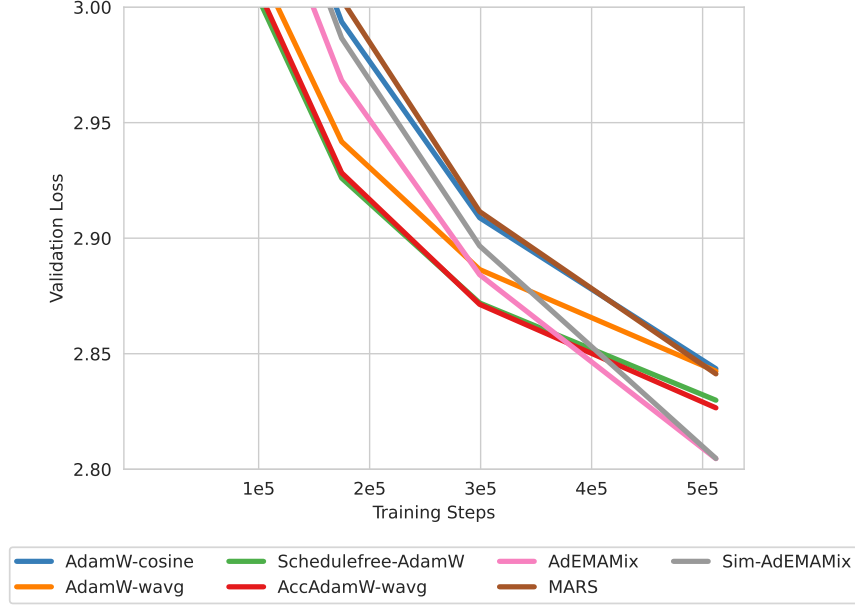


Figure 1: Comparison of the best runs of the various optimizers described in Section 5 on a decoder-only transformer language modeling task. Across the compared methods, AdEMAMix and Simplified-AdEMAMix consistently perform best, which we attribute to their close correspondence with accelerated SGD variants.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Full details of the hyperparameter sweeps for these algorithms are provided in Section A.

As illustrated in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably, directly supporting the equivalence predicted by our theory. Both, in turn, outperform AdamW with cosine decay and AdamW with tailed weight averaging. Most notably, AdEMAMix and Simplified-AdEMAMix are

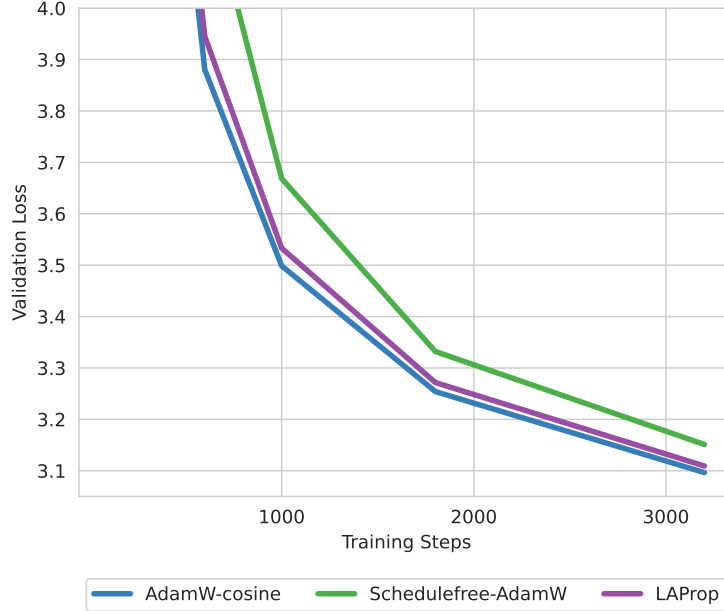


Figure 2: Comparison of the best runs of AdamW with cosine decay, Schedule-Free AdamW, and LARProp in the large-batch regime. Full experimental details are given in Section 5.1.

the strongest performers across the entire comparison—a pattern we attribute to their tight alignment with accelerated SGD variants, and one that is consistent across the optimizers and settings we examine.

5.1 LARGE BATCH SIZE EXPERIMENTS

The experiments above concern the small-batch, noise-dominated regime. To probe how broadly the conclusions hold, we now move to the opposite end of the spectrum—the large-batch regime, where gradient noise is much smaller—and ask whether these algorithms continue to help. Here we train with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale), giving us coverage of both extremes of the batch-size axis.

Schedule-Free AdamW: As shown in Figure 2, Schedule-Free AdamW performs markedly worse than AdamW in this regime. We attribute the gap to the coupling between the weight-averaging and momentum coefficients, a limitation that our framework predicts should surface precisely at large batch sizes. In this regime the optimal momentum value lies well below $1 - 1/t$; while one could compensate with a scaling factor $\approx 1 - r/t$ for some $r \geq 1$, any larger r shrinks the effective weight-averaging window, so the coupling cannot be resolved without sacrificing one benefit for the other.

A further difference between AdamW and Schedule-Free AdamW is the order in which momentum and preconditioning are applied: AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW does the reverse, making it algorithmically akin to LARProp (Ziyin et al., 2021). To isolate whether this ordering is responsible, we include LARProp in the comparison; as Figure 2 shows, AdamW and LARProp perform comparably, which rules out the ordering as the primary cause and points back to the coupling as the general explanation.

AdEMAMix: In the large-batch regime, and consistent with Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix causes a substantial drop relative to keeping two separate momentum terms. As anticipated by our earlier analysis, this degradation arises because AdEMAMix fixes the weight on the current gradient at 1, whereas the theory of accelerated SGD (Gupta et al., 2023) calls for a diminishing gradient weight as the batch size grows—so the same principle explains behavior at both ends of the batch-size range.

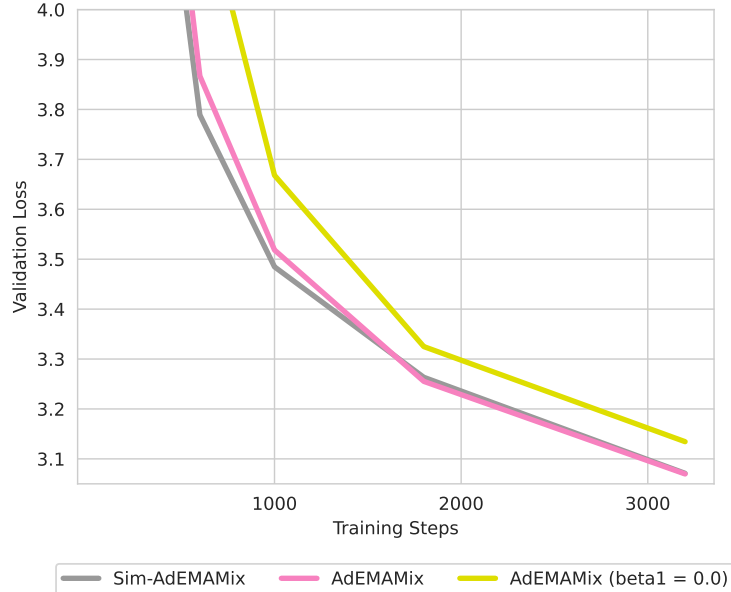


Figure 3: Comparison of the best runs of AdEMAMix (with and without $\beta_1 = 0.0$) and our Simplified-AdEMAMix variant in the large-batch regime. Full experimental details are given in Section 5.1.

Figure 3 also shows that our Simplified-AdEMAMix matches AdEMAMix in this regime while using only a single momentum term. Strikingly, it does so at $\alpha = 0.0$, meaning that in the large-batch setting Simplified-AdEMAMix reduces to standard Adam with momentum scheduling—an outcome that neatly connects the whole family back to a familiar baseline.

6 CONCLUSION

In this work, we established explicit and unifying connections between accelerated SGD variants and a broad family of recently proposed optimizers—including Schedule-Free optimizers, AdEMAMix, MARS, and Lion—showing that methods introduced from very different motivations can be read as instances of a single underlying framework. Within this view, we presented empirical evidence that AdEMAMix, the method that aligns most closely with theoretical accelerated SGD variants, achieves the strongest performance in small-batch training.

Building on this connection, we introduced Simplified-AdEMAMix, which does away with the need to maintain two separate momentum buffers. We showed empirically that it matches AdEMAMix across the range of batch sizes we study, from small to large, while removing the additional memory overhead. More broadly, we hope the framework developed here offers a common vocabulary for understanding current momentum-based optimizers and a principled guide for designing future ones; extending the analysis to further architectures, data domains, and training scales is a natural direction for future work.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

We report below the full hyperparameter configurations swept for the small-batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The corresponding hyperparameter sweeps for the large-batch experiments are reported below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

The general accelerated SGD form is given in Equation (1). In this section we show, method by method, that the accelerated schemes of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) all reduce to this single form, underscoring how widely the framework applies across the proposals in the literature.

B.1 AGNES

The update for Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the objective is evaluated at x_n . These equations can be rearranged into

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad - \frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(- \frac{v_n}{\rho_{n-1}} \right) + g'_n$$

so that x'_{n+1} obeys an update of exactly the form of Equation (1).

B.2 ASGD

The update for Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the objective is evaluated at the tail-averaged x . These update equations can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

and these again match the general form of Equation (1).

B.3 MASS

The update for Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the objective is evaluated at w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

and these again match the general form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

which, once more, can be rearranged into the general form:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$